Danmarks Tekniske Universitet

34349 FPGA Design for Communication Systems

Exercise 1 – Ethernet FCS

Student s224017

March 2, 2026

Exercise 1

Repository: https://github.com/amal-araweelo/34349_fpga_comm_exercises/tree/main/ex1_ethernet_fcs

Task 1 – Design FCS checker (serial)

Files

File name	Listing #
`/rtl/fcs_serial_check.sv`	Listing 1

Task 2 – Synthesis, Place & Route and Timing Analysis

Device and Tool Configuration

Quartus Version	25.1std.0 Build 1129 (SC Lite)
Device Family	MAX 10
Device	10M50DAF484C7G
Top-level Entity	`fcs_serial_check`

Resource Utilization

Resource	Used	Available
Logic Elements	97	49,760
Registers	48	–
Pins	6	360
Memory Bits	0	1,677,312
Multipliers	0	288
PLLs	0	4

Timing Analysis

Clock	Fmax (MHz)	Restricted Fmax (MHz)	Limitation
clk	287.85	250.00	Minimum period constraint

Task 3 – Simulate Design (serial)

Files

File name	Type	Listing #
`/tb/tb_fcs_serial_check.sv`	Test bench	Listing 5
`/tb/src/fcs_if.sv`	Interface	Listing 3
`/tb/src/fcs_pkg.sv`	Package	Listing 4

Task 4 – Design FCS checker, simulate design, Synthesis, Place & Route and Timing Analysis (parallel)

Files

File name	Type	Listing #
`/rtl/fcs_parallel_check.sv`	RTL	Listing 2
`/tb/tb_fcs_parallel_check.sv`	Testbench	Listing 6
`/tb/src/fcs_if.sv`	Interface	Listing 3
`/tb/src/fcs_pkg.sv`	Package	Listing 4

8-bit parallel CRC

To derive the 8-bit parallel CRC, the serial bitwise recursion is written as a linear state-space system over GF(2). Define the state vector as the concatenation of the CRC register and the 8-bit input window

$$\mathbf{x} = \begin{bmatrix} \mathbf{R} \\ \mathbf{M} \end{bmatrix} = \begin{bmatrix} R_{31} & \dots & R_0 & M_0 & \dots & M_7 \end{bmatrix}^T .$$

The serial update can then be expressed as

$$\mathbf{x}_{k+1} = \mathbf{A}\mathbf{x}_k \pmod 2,$$

where $\mathbf{A} \in \mathbb{F}_2^{40\times 40}$ is the state transition matrix.

The matrix $\mathbf{A}$ is constructed by expressing each next-state bit as a linear combination (XOR) of the current CRC and input bits. In block form,

$$\mathbf{A} = \begin{bmatrix} \mathbf{A}_{GI} & \mathbf{A}_Z \\ \mathbf{A}_C & \mathbf{A}_R \end{bmatrix},$$

where:

- $\mathbf{A}_{GI} \in \mathbb{F}_2^{32\times 32}$ includes the generator polynomial in the last row, an identity matrix $\mathbf{I}_{31}$ in the upper left corner. The rest of the entries in this matrix are 0.
- $\mathbf{A}_Z \in \mathbb{F}_2^{32\times 8}$ is a zero matrix.
- $\mathbf{A}_R \in \mathbb{F}_2^{8\times 8}$ is the right rotation matrix describing the shift of the 8-bit input data.
- $\mathbf{C} \in \mathbb{F}_2^{8\times 32}$ is a corner matrix.

The 8-bit parallel implementation is then obtained by computing

$$\mathbf{x}_{k+8} = \mathbf{A}^8\mathbf{x}_k \pmod 2,$$

so that $\mathbf{A}^8$ directly defines the parallel CRC update equations. I exported the result of $\mathbf{A}^8$ as an excel sheet, and asked ChatGPT to produce a script that could produce the equations in systemVerilog format:

```
// ------------------------------
// Auto-generated CRC equations
// ------------------------------

crc_r[0] <= crc_r[24] ^ crc_r[30] ^ data_in[7];
crc_r[1] <= crc_r[24] ^ crc_r[25] ^ crc_r[30] ^ crc_r[31] ^ data_in[6];
crc_r[2] <= crc_r[24] ^ crc_r[25] ^ crc_r[26] ^ crc_r[30] ^ crc_r[31] ^ data_in[5];
crc_r[3] <= crc_r[25] ^ crc_r[26] ^ crc_r[27] ^ crc_r[31] ^ data_in[4];
crc_r[4] <= crc_r[24] ^ crc_r[26] ^ crc_r[27] ^ crc_r[28] ^ crc_r[30] ^ data_in[3];
crc_r[5] <= crc_r[24] ^ crc_r[25] ^ crc_r[27] ^ crc_r[28] ^ crc_r[29] ^ crc_r[30] ^
    crc_r[31] ^ data_in[2];
crc_r[6] <= crc_r[25] ^ crc_r[26] ^ crc_r[28] ^ crc_r[29] ^ crc_r[30] ^ crc_r[31] ^
    data_in[1];
crc_r[7] <= crc_r[24] ^ crc_r[26] ^ crc_r[27] ^ crc_r[29] ^ crc_r[31] ^ data_in[0];
crc_r[8] <= crc_r[0] ^ crc_r[24] ^ crc_r[25] ^ crc_r[27] ^ crc_r[28];
crc_r[9] <= crc_r[1] ^ crc_r[25] ^ crc_r[26] ^ crc_r[28] ^ crc_r[29];
crc_r[10] <= crc_r[2] ^ crc_r[24] ^ crc_r[26] ^ crc_r[27] ^ crc_r[29];
crc_r[11] <= crc_r[3] ^ crc_r[24] ^ crc_r[25] ^ crc_r[27] ^ crc_r[28];
crc_r[12] <= crc_r[4] ^ crc_r[24] ^ crc_r[25] ^ crc_r[26] ^ crc_r[28] ^ crc_r[29] ^
    crc_r[30];
crc_r[13] <= crc_r[5] ^ crc_r[25] ^ crc_r[26] ^ crc_r[27] ^ crc_r[29] ^ crc_r[30] ^
    crc_r[31];
crc_r[14] <= crc_r[6] ^ crc_r[26] ^ crc_r[27] ^ crc_r[28] ^ crc_r[30] ^ crc_r[31];
crc_r[15] <= crc_r[7] ^ crc_r[27] ^ crc_r[28] ^ crc_r[29] ^ crc_r[31];
crc_r[16] <= crc_r[8] ^ crc_r[24] ^ crc_r[28] ^ crc_r[29];
crc_r[17] <= crc_r[9] ^ crc_r[25] ^ crc_r[29] ^ crc_r[30];
crc_r[18] <= crc_r[10] ^ crc_r[26] ^ crc_r[30] ^ crc_r[31];
crc_r[19] <= crc_r[11] ^ crc_r[27] ^ crc_r[31];
crc_r[20] <= crc_r[12] ^ crc_r[28];
crc_r[21] <= crc_r[13] ^ crc_r[29];
crc_r[22] <= crc_r[14] ^ crc_r[24];
crc_r[23] <= crc_r[15] ^ crc_r[24] ^ crc_r[25] ^ crc_r[30];
crc_r[24] <= crc_r[16] ^ crc_r[25] ^ crc_r[26] ^ crc_r[31];
crc_r[25] <= crc_r[17] ^ crc_r[26] ^ crc_r[27];
crc_r[26] <= crc_r[18] ^ crc_r[24] ^ crc_r[27] ^ crc_r[28] ^ crc_r[30];
crc_r[27] <= crc_r[19] ^ crc_r[25] ^ crc_r[28] ^ crc_r[29] ^ crc_r[31];
crc_r[28] <= crc_r[20] ^ crc_r[26] ^ crc_r[29] ^ crc_r[30];
crc_r[29] <= crc_r[21] ^ crc_r[27] ^ crc_r[30] ^ crc_r[31];
crc_r[30] <= crc_r[22] ^ crc_r[28] ^ crc_r[31];
crc_r[31] <= crc_r[23] ^ crc_r[29];
data_in[0 ] <= data_in[0];
data_in[1] <= data_in[1];
data_in[2] <= data_in[2];
data_in[3] <= data_in[3];
data_in[4] <= data_in[4];
data_in[5] <= data_in[5];
data_in[6] <= data_in[6];
data_in[7] <= data_in[7];
```

It was at this point that I realized I had made a mistake in the definition of the **M**-vector. The LSB should have been M_0 and the MSB, M_7. Fixing this (see Listing 2), the CRC-checker for the parallel implementation worked.

Device and Tool Configuration

Quartus Version	25.1std.0 Build 1129 (SC Lite)
Device Family	MAX 10
Device	10M50DAF484C7G
Top-level Entity	`fcs_parallel_check`

Resource Utilization

Resource	Used	Available
Logic Elements	101	49,760
Registers	41	–
Pins	13	360
Memory Bits	0	1,677,312
Multipliers	0	288
PLLs	0	4

Timing Analysis

Clock	Fmax (MHz)	Restricted Fmax (MHz)	Limitation
clk	300.66	250.00	Minimum period constraint

At first glance, the higher $F_{\max}$ of the parallel implementation appears counterintuitive, since the 8-bit update introduces significantly more combinational XOR logic compared to the serial version. I initially therefore expected a longer critical path.

However, inspection of the timing report shows that the critical path is not dominated by the CRC combinational network, but by the control logic, in particular the frame and FCS counters. In the serial design, wider counters are required (6 bits would have sufficed, but I wrongly used 7 bits), whereas in the parallel implementation the counters operate on bytes and are therefore narrower (2 bits would have sufficed, but I again used 4 bits).

The reduced counter width shortens the arithmetic and comparison logic in the critical path, resulting in a slightly higher achievable $F_{\max}$ for the parallel implementation.

Systemverilog code

RTL files

Listing 1: Serial FCS checker

```
module fcs_serial_check (
    input  logic clk,              // system clock
    input  logic reset,            // synchronous, active low reset
    input  logic start_of_frame,   // arrival of the first bit.
    input  logic end_of_frame,     // arrival of the first bit in FCS.
    input  logic data_in,          // serial input data.
    output logic fcs_error        // indicates an error.
  );

  logic [31:0] crc_r;
  logic        in_frame;
  logic [6:0]  start_cntr;
  logic [6:0]  fcs_bit_cntr;
  logic        fcs_error_r;
  logic fb;


  // Compute feedback bit
  always_comb begin
    if (start_cntr < 'd32 && start_cntr != 'd0) begin
      fb = ~data_in ^ crc_r[31];  // negate payload bits
    end else if (start_of_frame) begin
      fb = ~data_in ^ crc_r[31]; // negate first bit of payload
    end else if (fcs_bit_cntr < 'd32) begin
      fb = ~data_in ^ crc_r[31]; // negate FCS bits
    end else if (end_of_frame) begin
      fb = ~data_in ^ crc_r[31]; // negate first bit of FCS
    end else begin
      fb = data_in ^ crc_r[31];  // no negation for the rest
    end
  end

  // Ouput assignment
  assign fcs_error = fcs_error_r; // pulses for one clock cycle at the end of frame
      if there is an error

  always_ff @(posedge clk) begin

    fcs_error_r <= 'b0; // default to no error, will be set at the end of frame

    if (!reset) begin
      crc_r        <= 'b0;
      start_cntr   <= 'd32;
      fcs_bit_cntr <= 'd32;
      in_frame     <= 'b0;
      fcs_error_r  <= 'b0;
    end else begin
```

```
// Start of frame (reset counter and enable in frame)
if (start_of_frame) begin
    in_frame <= 'b1;
    start_cntr <= 'd31;
end

// End of frame
if (end_of_frame) begin
  fcs_bit_cntr <= fcs_bit_cntr - 'd1;
end

// Generator polynomial: x^26 + x^23 + x^22 + x^16 + x^12 + x^11 + x^10 + x^8 +
    x^7 + x^5 + x^4 + x^2 + x + 1
if (start_of_frame || in_frame) begin // while processing frame
  crc_r[31] <= crc_r[30]; // CRC shift register
  crc_r[30] <= crc_r[29];
  crc_r[29] <= crc_r[28];
  crc_r[28] <= crc_r[27];
  crc_r[27] <= crc_r[26];
  crc_r[26] <= crc_r[25] ^ crc_r[31];
  crc_r[25] <= crc_r[24];
  crc_r[24] <= crc_r[23];
  crc_r[23] <= crc_r[22] ^ crc_r[31];
  crc_r[22] <= crc_r[21] ^ crc_r[31];
  crc_r[21] <= crc_r[20];
  crc_r[20] <= crc_r[19];
  crc_r[19] <= crc_r[18];
  crc_r[18] <= crc_r[17];
  crc_r[17] <= crc_r[16];
  crc_r[16] <= crc_r[15] ^ crc_r[31];
  crc_r[15] <= crc_r[14];
  crc_r[14] <= crc_r[13];
  crc_r[13] <= crc_r[12];
  crc_r[12] <= crc_r[11] ^ crc_r[31];
  crc_r[11] <= crc_r[10] ^ crc_r[31];
  crc_r[10] <= crc_r[9]  ^ crc_r[31];
  crc_r[9]  <= crc_r[8];
  crc_r[8]  <= crc_r[7] ^ crc_r[31];
  crc_r[7]  <= crc_r[6] ^ crc_r[31];
  crc_r[6]  <= crc_r[5];
  crc_r[5]  <= crc_r[4] ^ crc_r[31];
  crc_r[4]  <= crc_r[3] ^ crc_r[31];
  crc_r[3]  <= crc_r[2];
  crc_r[2]  <= crc_r[1] ^ crc_r[31];
  crc_r[1]  <= crc_r[0] ^ crc_r[31];
  crc_r[0]  <= fb;

  // Decrement start bit counter only when we are in the payload part of the
      frame
  if (start_cntr < 'd32 && start_cntr != 'd0) start_cntr <= start_cntr - 'd1;
  else if (start_cntr == 'd1) start_cntr <= 'd0; // keep at 0

  // Decrement FCS bit counter only when we are in the FCS part of the frame
```

```
        if (fcs_bit_cntr < 'd32) fcs_bit_cntr <= fcs_bit_cntr - 'd1;
        else if (fcs_bit_cntr == 'd1) fcs_bit_cntr <= 'd0;

        // Check for FCS error at the end of frame
        if (fcs_bit_cntr == 'd0) begin
          fcs_error_r <= (crc_r == 32'h0000_0000) ? 'b0: 'b1;
          in_frame <='b0;

          // prepare for next frame
          crc_r <= '0;
          start_cntr   <= 'd32;
          fcs_bit_cntr <= 'd32;
        end
      end
    end
  end
endmodule
```

Listing 2: Parallel FCS checker

```
module fcs_parallel_check (
    input  logic        clk,
    input  logic        reset,
    input  logic        start_of_frame,
    input  logic        end_of_frame,
    input  logic [7:0]  data_in,
    output logic        fcs_error
);

  logic [31:0] crc_r;

  logic        in_frame;
  logic [3:0]  start_cntr;      // 32 bits = 4 bytes
  logic [3:0]  fcs_byte_cntr;   // 4 FCS bytes
  logic        fcs_error_r;

  logic [7:0]  data_proc;

  // Ouput assignment
  assign fcs_error = fcs_error_r;

  // Compute feedback bits
  always_comb begin
    data_proc = data_in; // default to no negation
    if (start_cntr < 4 && start_cntr != 0)
      data_proc = ~data_in;  // negate payload byte
    else if (start_of_frame)
      data_proc = ~data_in; // negate first byte of payload
    else if (fcs_byte_cntr < 4)
      data_proc = ~data_in; // negate FCS byte
    else if (end_of_frame)
      data_proc = ~data_in; // negate first byte of FCS
  end
```

```
always_ff @(posedge clk) begin

  fcs_error_r <= 1'b0; // pulses for one clock cycle at the end of frame if there
      is an error

  if (!reset) begin
    crc_r          <= 32'b0;
    start_cntr     <= 4;
    fcs_byte_cntr  <= 4;
    in_frame       <= 1'b0;
    fcs_error_r    <= 1'b0;
  end
  else begin
    // Start of frame (reset counter and enable in frame)
    if (start_of_frame) begin
      in_frame    <= 1'b1;
      start_cntr  <= 3;
    end

    if (end_of_frame)
      fcs_byte_cntr <= fcs_byte_cntr - 1;

    if (start_of_frame || in_frame) begin // while processing frame

      // 8-bit parallel update
      crc_r[0] <= crc_r[24] ^ crc_r[30] ^ data_proc[0];
      crc_r[1] <= crc_r[24] ^ crc_r[25] ^ crc_r[30] ^ crc_r[31] ^ data_proc[1];
      crc_r[2] <= crc_r[24] ^ crc_r[25] ^ crc_r[26] ^ crc_r[30] ^ crc_r[31] ^
          data_proc[2];
      crc_r[3] <= crc_r[25] ^ crc_r[26] ^ crc_r[27] ^ crc_r[31] ^ data_proc[3];
      crc_r[4] <= crc_r[24] ^ crc_r[26] ^ crc_r[27] ^ crc_r[28] ^ crc_r[30] ^
          data_proc[4];
      crc_r[5] <= crc_r[24] ^ crc_r[25] ^ crc_r[27] ^ crc_r[28] ^ crc_r[29] ^ crc_r
          [30] ^ crc_r[31] ^ data_proc[5];
      crc_r[6] <= crc_r[25] ^ crc_r[26] ^ crc_r[28] ^ crc_r[29] ^ crc_r[30] ^ crc_r
          [31] ^ data_proc[6];
      crc_r[7] <= crc_r[24] ^ crc_r[26] ^ crc_r[27] ^ crc_r[29] ^ crc_r[31] ^
          data_proc[7];
      crc_r[8] <= crc_r[0] ^ crc_r[24] ^ crc_r[25] ^ crc_r[27] ^ crc_r[28];
      crc_r[9] <= crc_r[1] ^ crc_r[25] ^ crc_r[26] ^ crc_r[28] ^ crc_r[29];
      crc_r[10] <= crc_r[24] ^ crc_r[26] ^ crc_r[27] ^ crc_r[29] ^ crc_r[2];
      crc_r[11] <= crc_r[24] ^ crc_r[25] ^ crc_r[27] ^ crc_r[28] ^ crc_r[3];
      crc_r[12] <= crc_r[24] ^ crc_r[25] ^ crc_r[26] ^ crc_r[28] ^ crc_r[29] ^
          crc_r[30] ^ crc_r[4];
      crc_r[13] <= crc_r[25] ^ crc_r[26] ^ crc_r[27] ^ crc_r[29] ^ crc_r[30] ^
          crc_r[31] ^ crc_r[5];
      crc_r[14] <= crc_r[26] ^ crc_r[27] ^ crc_r[28] ^ crc_r[30] ^ crc_r[31] ^
          crc_r[6];
      crc_r[15] <= crc_r[27] ^ crc_r[28] ^ crc_r[29] ^ crc_r[31] ^ crc_r[7];
      crc_r[16] <= crc_r[24] ^ crc_r[28] ^ crc_r[29] ^ crc_r[8];
      crc_r[17] <= crc_r[25] ^ crc_r[29] ^ crc_r[30] ^ crc_r[9];
```

```
        crc_r[18] <= crc_r[10] ^ crc_r[26] ^ crc_r[30] ^ crc_r[31];
        crc_r[19] <= crc_r[11] ^ crc_r[27] ^ crc_r[31];
        crc_r[20] <= crc_r[12] ^ crc_r[28];
        crc_r[21] <= crc_r[13] ^ crc_r[29];
        crc_r[22] <= crc_r[14] ^ crc_r[24];
        crc_r[23] <= crc_r[15] ^ crc_r[24] ^ crc_r[25] ^ crc_r[30];
        crc_r[24] <= crc_r[16] ^ crc_r[25] ^ crc_r[26] ^ crc_r[31];
        crc_r[25] <= crc_r[17] ^ crc_r[26] ^ crc_r[27];
        crc_r[26] <= crc_r[18] ^ crc_r[24] ^ crc_r[27] ^ crc_r[28] ^ crc_r[30];
        crc_r[27] <= crc_r[19] ^ crc_r[25] ^ crc_r[28] ^ crc_r[29] ^ crc_r[31];
        crc_r[28] <= crc_r[20] ^ crc_r[26] ^ crc_r[29] ^ crc_r[30];
        crc_r[29] <= crc_r[21] ^ crc_r[27] ^ crc_r[30] ^ crc_r[31];
        crc_r[30] <= crc_r[22] ^ crc_r[28] ^ crc_r[31];
        crc_r[31] <= crc_r[23] ^ crc_r[29];

        // Decrement start and FCS byte counters
        if (start_cntr < 4 && start_cntr != 0) // when processing first 4 payload
            bytes
          start_cntr <= start_cntr - 1;

        if (fcs_byte_cntr < 4)  // when processing 4 FCS bytes
          fcs_byte_cntr <= fcs_byte_cntr - 1;

        if (fcs_byte_cntr == 0) begin // after processing all 4 FCS bytes, check if
            crc is correct and pulse fcs_error if not
          fcs_error_r <= (crc_r == 32'h0000_0000) ? 1'b0 : 1'b1;
          in_frame <= 1'b0;

          // prepare for next frame
          crc_r         <= 32'b0;
          start_cntr    <= 4;
          fcs_byte_cntr <= 4;
        end

      end
    end
  end

endmodule
```

TB files

Listing 3: FCS interface

```
interface fcs_if #(
  parameter int DATA_WIDTH = 8)
  (
    input logic clk
  );

  logic reset;
  logic start_of_frame;
  logic end_of_frame;
```

```
  logic [DATA_WIDTH-1:0] data_in;
  logic fcs_error;


  clocking cb @(posedge clk); // all outputs -> inputs and vice versa
    output reset;
    output start_of_frame;
    output end_of_frame;
    output data_in;
    input  fcs_error;
  endclocking

endinterface
```

Listing 4: FCS package (class)

```
package fcs_pkg;

  class fcs_class;

    virtual fcs_if vif;

    function new(virtual fcs_if vif);
      this.vif = vif;
    endfunction

    // change for ethernet realistic payload
    localparam int PAYLOAD_LEN = 480; // payload length in bits (e.g., 60 bytes for
        Ethernet)

    ////////////////////// GENERATE PALOAD ////////////////////////////////
    function logic [PAYLOAD_LEN-1:0] generate_payload();
      logic [PAYLOAD_LEN-1:0] payload;

      // randomize payload with 0s and 1s
      for (int i = 0; i < PAYLOAD_LEN; i++)
        payload[i] = $urandom_range(0,1);

      return payload;
    endfunction


    ////////////////////// CALCULATE CRC ////////////////////////////////////
    function logic [31:0] calc_crc(
        logic [PAYLOAD_LEN-1:0] payload
      );
      logic [31:0]  crc;
      logic         fb; // feedback bit

      crc = 32'hFFFFFFFF;
      for (int i = PAYLOAD_LEN-1; i >= 0; i--) begin // Process each bit from MSB to
          LSB
          fb = payload[i] ^ crc[31];
```

```
          crc = {crc[30:0], 1'b0};
          if (fb)
              crc ^= 32'h04C11DB7;
      end
      return ~crc;
    endfunction

    ////////////////////////// RESET SEQUENCE ////////////////////////////////
    task automatic reset_seq();
      vif.cb.reset <= 'b0;
      repeat(5) @(vif.cb);
      vif.cb.reset <='b1;
    endtask
  endclass

endpackage
```

Listing 5: Serial FCS checker TB

```
import fcs_pkg::*;

module tb_fcs_serial_check;

  // Clock
  logic clk = 0;
  localparam int CLK_PERIOD = 10;

  always #(CLK_PERIOD/2) clk = ~clk;

  // Interface and class
  fcs_if i_fcs_if (clk);
  fcs_class fcs;

  initial
    fcs = new(i_fcs_if);

  // DUT instance
  fcs_serial_check dut (
                      .clk(i_fcs_if.clk),
                      .reset(i_fcs_if.reset),
                      .start_of_frame(i_fcs_if.start_of_frame),
                      .end_of_frame(i_fcs_if.end_of_frame),
                      .data_in(i_fcs_if.data_in),
                      .fcs_error(i_fcs_if.fcs_error)
                    );

  // Signals
  logic [fcs.PAYLOAD_LEN-1:0] payload;
  logic [31:0] crc;
  logic [31:0] crc_neg;
  logic [fcs.PAYLOAD_LEN-1:0] payload_neg;
  logic [fcs.PAYLOAD_LEN+31:0] frame;
```

```
////////////////////////////// TEST TASKS ////////////////////////////////
task automatic test_fixed_payload();

  // Fixed payload
  payload = 512'
      h0010_A47B_EA80_0012_3456_7890_0800_4500_002E_B3FE_0000_8011_0540_C0A8_002C_C0A8_0004_0400
      ; // fixed payload for debugging
  // payload_neg = {~payload[479:448],payload[447:0]}; // only complement the MSB
      32 bits of the payload (for debugging)
  crc = 32'hE6C53DB2;
  // crc_neg = ~crc; // for debugging
  frame = {payload, crc}; // frame to be fed to fcs_serial_check

  // Send serially
  i_fcs_if.cb.start_of_frame <= 'b1; // pulse start of frame
  i_fcs_if.cb.data_in <= frame[fcs.PAYLOAD_LEN + 31]; // first bit of payload with
      start of frame
  @(i_fcs_if.cb);
  i_fcs_if.cb.start_of_frame <= 'b0;

  // Remaining bits
  for (int i = fcs.PAYLOAD_LEN + 31 - 1; i >= 0; i--) begin
    i_fcs_if.cb.data_in      <= frame[i];  // feed bits from MSB to LSB
    i_fcs_if.cb.end_of_frame <= (i == 31); // first FCS bit
    @(i_fcs_if.cb);
    i_fcs_if.cb.end_of_frame <= 'b0;
  end
  @(i_fcs_if.cb); // wait one clock cycle for the final crc register update

  @(i_fcs_if.cb); // wait another cycle for fcs_error to be pulsed

  // Check result
  if (i_fcs_if.cb.fcs_error == 'b0)begin
    $display("TEST 2 (FIXED PAYLOAD, CORRECT FCS HEADER): PASS");
  end else begin
    $display("TEST 2 (FIXED PAYLOAD, CORRECT FCS HEADER): FAIL \n");
    $display("got fcs_error: b%b, expected fcs_error = b0", i_fcs_if.cb.fcs_error);
  end
endtask

task automatic test_fixed_payload_wrong_crc();
    // Fixed payload
  payload = 512'
      h0010_A47B_EA80_0012_3456_7890_0800_4500_002E_B3FE_0000_8011_0540_C0A8_002C_C0A8_0004_0400
      ; // fixed payload for debugging
  crc = 32'hE6C53DB1;
  frame = {payload, crc}; // frame to be fed to fcs_serial_check

  // Send serially
  i_fcs_if.cb.start_of_frame <= 'b1; // pulse start of frame
  i_fcs_if.cb.data_in <= frame[fcs.PAYLOAD_LEN + 31]; // first bit of payload with
```

```
    start of frame
  @(i_fcs_if.cb);
  i_fcs_if.cb.start_of_frame <= 'b0;

  // Send all remaining bits
  for (int i = fcs.PAYLOAD_LEN + 31 - 1; i >= 0; i--) begin
    i_fcs_if.cb.data_in      <= frame[i];  // feed bits from MSB to LSB
    i_fcs_if.cb.end_of_frame <= (i == 31); // first FCS bit
    @(i_fcs_if.cb);
    i_fcs_if.cb.end_of_frame <= 'b0;
  end
  @(i_fcs_if.cb); // wait one clock cycle for the final crc register update

  @(i_fcs_if.cb); // wait another cycle for fcs_error to be pulsed

  // Check result
  if (i_fcs_if.cb.fcs_error == 'b1)begin
    $display("TEST 2 (FIXED PAYLOAD, WRONG FCS HEADER): PASS");
  end else begin
    $display("TEST 2 (FIXED PAYLOAD, WRONG FCS HEADER): FAIL");
    $display("got fcs_error: b%b, expected fcs_error = b1", i_fcs_if.cb.fcs_error);
  end
endtask

task automatic test_random_payload();
  // Fixed payload
  payload = fcs.generate_payload(); // random payload for testing
  crc = fcs.calc_crc(payload);
  frame = {payload, crc}; // frame to be fed to fcs_serial_check

  // Send serially
  i_fcs_if.cb.start_of_frame <= 'b1; // pulse start of frame
  i_fcs_if.cb.data_in <= frame[fcs.PAYLOAD_LEN + 31]; // first bit of payload with
    start of frame
  @(i_fcs_if.cb);
  i_fcs_if.cb.start_of_frame <= 'b0;

  // Remaining bits
  for (int i = fcs.PAYLOAD_LEN + 31 - 1; i >= 0; i--) begin
    i_fcs_if.cb.data_in      <= frame[i];  // feed bits from MSB to LSB
    i_fcs_if.cb.end_of_frame <= (i == 31); // first FCS bit
    @(i_fcs_if.cb);
    i_fcs_if.cb.end_of_frame <= 'b0;
  end
  @(i_fcs_if.cb); // wait one clock cycle for the final crc register update

  @(i_fcs_if.cb); // wait another cycle for fcs_error to be pulsed

  // Check result
  if (i_fcs_if.cb.fcs_error == 'b0)begin
    $display("TEST 3 (RANDOM PAYLOAD, CORRECT FCS HEADER): PASS");
  end else begin
    $display("TEST 3 (RANDOM PAYLOAD, CORRECT FCS HEADER): FAIL \n");
```

```
      $display("got fcs_error: b%b, expected fcs_error = b0", i_fcs_if.cb.fcs_error);
    end
  endtask

  task automatic test_random_payload_wrong_crc();
    // Fixed payload
    payload = fcs.generate_payload(); // random payload for testing
    crc = fcs.calc_crc(payload);
    crc[31:30] = ~crc[31:30]; // introduce error in the FCS
    frame = {payload, crc}; // frame to be fed to fcs_serial_check

    // Send serially
    i_fcs_if.cb.start_of_frame <= 'b1; // pulse start of frame
    i_fcs_if.cb.data_in <= frame[fcs.PAYLOAD_LEN + 31]; // first bit of payload with
        start of frame
    @(i_fcs_if.cb);
    i_fcs_if.cb.start_of_frame <= 'b0;

    // Remaining bits
    for (int i = fcs.PAYLOAD_LEN + 31 - 1; i >= 0; i--) begin
      i_fcs_if.cb.data_in      <= frame[i];  // feed bits from MSB to LSB
      i_fcs_if.cb.end_of_frame <= (i == 31); // first FCS bit
      @(i_fcs_if.cb);
      i_fcs_if.cb.end_of_frame <= 'b0;
    end
    @(i_fcs_if.cb); // wait one clock cycle for the final crc register update

    @(i_fcs_if.cb); // wait another cycle for fcs_error to be pulsed

    // Check result
    if (i_fcs_if.cb.fcs_error == 'b1)begin
      $display("TEST 4 (RANDOM PAYLOAD, WRONG FCS HEADER): PASS");
    end else begin
      $display("TEST 4 (RANDOM PAYLOAD, WRONG FCS HEADER): FAIL \n");
      $display("got fcs_error: b%b, expected fcs_error = b1", i_fcs_if.cb.fcs_error);
    end
  endtask


  ///////////////////////////// RUN ALL TESTS ////////////////////////////
  initial begin // run tests
    fcs.reset_seq();
    test_fixed_payload();
    test_fixed_payload_wrong_crc();
    test_random_payload();
    test_random_payload_wrong_crc();
    $stop();
  end
endmodule
```

Listing 6: Parallel FCS checker TB

```
import fcs_pkg::*;
```

```
module tb_fcs_parallel_check;

  // Clock
  logic clk = 0;
  localparam int CLK_PERIOD = 10;

  always #(CLK_PERIOD/2) clk = ~clk;

  // Interface and class
  fcs_if #(.DATA_WIDTH(8)) i_fcs_if (clk);
  fcs_class fcs;

  initial
    fcs = new(i_fcs_if);

  // DUT instance
  fcs_parallel_check dut (
    .clk(i_fcs_if.clk),
    .reset(i_fcs_if.reset),
    .start_of_frame(i_fcs_if.start_of_frame),
    .end_of_frame(i_fcs_if.end_of_frame),
    .data_in(i_fcs_if.data_in),
    .fcs_error(i_fcs_if.fcs_error)
  );

  // Signals
  logic [fcs.PAYLOAD_LEN-1:0] payload;
  logic [fcs.PAYLOAD_LEN-1:0] payload_neg;
  logic [31:0] crc;
  logic [31:0] crc_neg;
  logic [fcs.PAYLOAD_LEN+31:0] frame;

  ////////////////////////////// TEST TASKS ////////////////////////////////
  task automatic send_frame();
    int total_bits = fcs.PAYLOAD_LEN + 32;

    // Send first byte with start_of_frame
    i_fcs_if.cb.start_of_frame <= 1'b1;
    i_fcs_if.cb.data_in <= frame[total_bits-1 -: 8];
    @(i_fcs_if.cb);
    i_fcs_if.cb.start_of_frame <= 1'b0;

    // Remaining bytes
    for (int i = total_bits-8-1; i >= 0; i -= 8) begin
      i_fcs_if.cb.data_in <= frame[i -: 8];
      i_fcs_if.cb.end_of_frame <= (i == 31); // first FCS byte
      @(i_fcs_if.cb);
      i_fcs_if.cb.end_of_frame <= 1'b0;
    end

    @(i_fcs_if.cb); // final CRC update
    @(i_fcs_if.cb); // allow fcs_error pulse
```

```
endtask

task automatic test_fixed_payload();

  payload = 512'
      h0010_A47B_EA80_0012_3456_7890_0800_4500_002E_B3FE_0000_8011_0540_C0A8_002C_C0A8_0004_0400
      ;
  crc = 32'hE6C53DB2;
  payload_neg = {~payload[479:448],payload[447:0]}; // only complement the MSB 32
      bits of the payload (for debugging)
  crc = 32'hE6C53DB2;
  crc_neg = ~crc; // for debugging
  frame = {payload, crc};

  send_frame();

  if (i_fcs_if.cb.fcs_error == 1'b0)
    $display("TEST 1 (FIXED PAYLOAD, CORRECT FCS): PASS");
  else begin
    $display("TEST 1 (FIXED PAYLOAD, CORRECT FCS): FAIL");
    $display("got fcs_error: %b", i_fcs_if.cb.fcs_error);
  end
endtask


task automatic test_fixed_payload_wrong_crc();

  payload = 512'
      h0010_A47B_EA80_0012_3456_7890_0800_4500_002E_B3FE_0000_8011_0540_C0A8_002C_C0A8_0004_0400
      ;
  crc = 32'hE6C53DB1;
  frame = {payload, crc};

  send_frame();

  if (i_fcs_if.cb.fcs_error == 1'b1)
    $display("TEST 2 (FIXED PAYLOAD, WRONG FCS): PASS");
  else begin
    $display("TEST 2 (FIXED PAYLOAD, WRONG FCS): FAIL");
    $display("got fcs_error: %b", i_fcs_if.cb.fcs_error);
  end
endtask

task automatic test_random_payload();

  payload = fcs.generate_payload();
  crc = fcs.calc_crc(payload);
  frame = {payload, crc};

  send_frame();

  if (i_fcs_if.cb.fcs_error == 1'b0)
    $display("TEST 3 (RANDOM PAYLOAD, CORRECT FCS): PASS");
```

```
    else begin
      $display("TEST 3 (RANDOM PAYLOAD, CORRECT FCS): FAIL");
      $display("got fcs_error: %b", i_fcs_if.cb.fcs_error);
    end
  endtask

  task automatic test_random_payload_wrong_crc();

    payload = fcs.generate_payload();
    crc = fcs.calc_crc(payload);
    crc[31:30] = ~crc[31:30];
    frame = {payload, crc};

    send_frame();

    if (i_fcs_if.cb.fcs_error == 1'b1)
      $display("TEST 4 (RANDOM PAYLOAD, WRONG FCS): PASS");
    else begin
      $display("TEST 4 (RANDOM PAYLOAD, WRONG FCS): FAIL");
      $display("got fcs_error: %b", i_fcs_if.cb.fcs_error);
    end
  endtask


  ///////////////////////////// RUN ALL TESTS ////////////////////////////////
  initial begin
    fcs.reset_seq();
    test_fixed_payload();
    test_fixed_payload_wrong_crc();
    test_random_payload();
    test_random_payload_wrong_crc();
    $stop();
  end

endmodule
```